

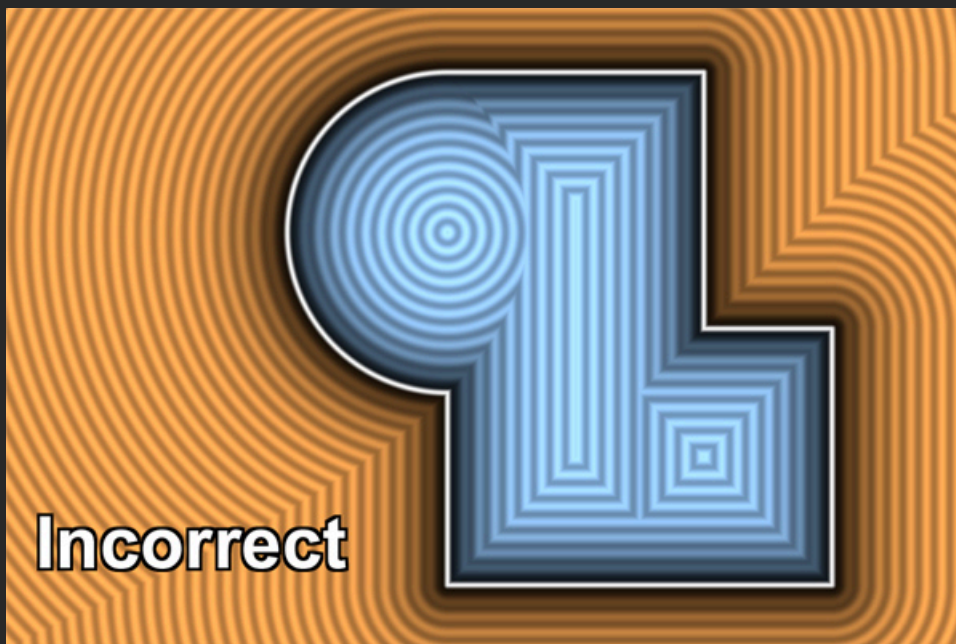
Inigo Quilez :: articles :: interior SDFs - 2020



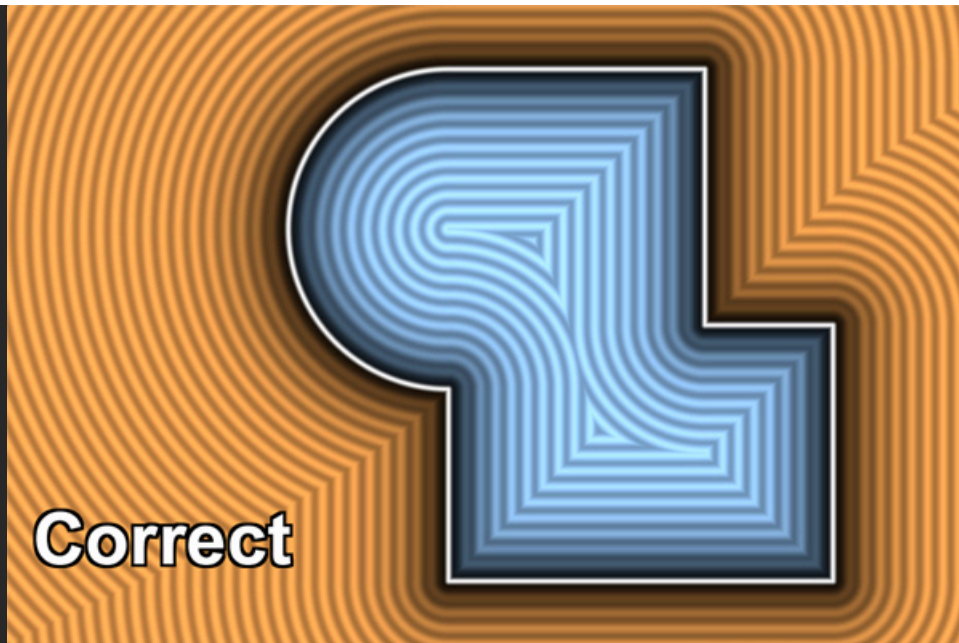
Intro

When modeling with SDFs, there are two ways to achieve complex shapes: one is to combine simpler shapes with union, subtraction and intersection operations, which are usually implemented with $\min()$ and $\max()$ functions. The other one is to design a new formula from scratch for the SDF of the desired shape. The second option involves some mathematical derivation, which is just fine with the caveat that sometimes it might be impossible to get such closed form formula. Because of that, people tend to work with a very small set of basic primitives and then just combine them as needed.

However, the $\min()$ and $\max()$ of two SDFs do not always produce a valid SDF, in that the result is actually not a distance field, but just a lower bound to the real distance to the resulting surface. This impacts the performance and quality of the algorithm used with the SDF (like in the case of a raymarcher), or can even completely break it and stop functioning (like in the case of collision detection).



Incorrect SDF - using $\min()$ to combine one circle and two boxes



Correct SDF

Now, while it is well known that the $\max()$ based operators (subtraction and intersection) break the SDF of the resulting shape (they produce bounds, not correct distance everywhere), the $\min()$ based ones (union) are often assumed to produce correct SDFs. But this is sometimes an incorrect assumption. In particular, in the interior of the resulting shape the SDF, where it takes negative values, it doesn't work as expected. I'm not talking about small deviation from the real SDF or accuracy problems, but of totally broken SDFs.

However, one might think that the interior of the SDF is often not needed, except for subsurface scattering or volumetric effects perhaps. But this is also not true. This is clear when modeling a shape or scene through its negative space. For example, one might desire to model the interior of a L-shaped room by using two box SDFs, making their union, and then flipping the sign of the SDF. After all, an L shape really looks like two boxes. However, the SDF produced by the union of two boxes and a negation is totally wrong, and neither the raymarcher in charge of rendering nor the physics engine will work with it as expected. The images bellow show the difference between the SDF generated with $\min()$ of two boxes and a disk, which is wrong, and the correct SDF for that shape. So, let's see what we can do about this problem.

Idea 1

The first thing one can do is of course to ignore the problem. That's totally alright as long as one knows the

implications and limitations that come with the decision.

For example, ignoring the problem won't backfire if we are raymarching only solid/opaque geometry. In principle a vanilla distance field raymarcher doesn't need to ever touch the inner volume of the shapes, so we should be good. Some more advanced raymarching techniques might, but let's assume we are using a basic raymarching setup for now.

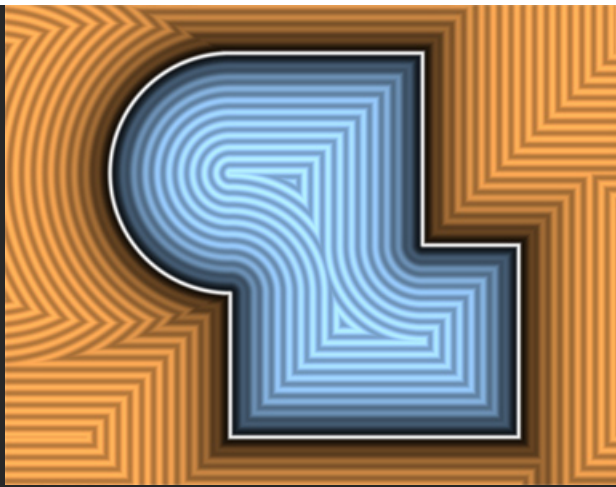
In theory the broken interior SDF could still create some problems during the numerical computation of the surface normals and/or gradient, which often is implemented as **central differences** that can potentially (and often do) sample the interior of the SDF. In practice, the artifacts introduced by `min()` exist mostly in the deep interior of the shape and only touch the surface at isolated locations. Since the central differences method usually sample in a small neighborhood of the surface, the errors are rare and small when they exist. And in practice, the problem can often just be ignored.

Idea 2

However, sometimes we don't want to ignore the interior of our SDF. This can be, as mentioned in the introduction, because we've realized that the interior of a given SDF is really close to the object we are modeling. For example, the L shaped room example above, made of two boxes and one circle, is very easily modeled with two boxes and one circle indeed. At least its walls or in/out interface is. The interior of the room is however broken due to the SDF problem that we are analyzing today.

One thing that we can do, that might work in some situations, is to reverse the problem and think of the negative space around the shape. In the case of this room, it would mean making our SDF model the outside of the room (the yellow areas) instead of the interior (the blue ones). If we think inside-out we can see that we can model the room by using five boxes and one arc/moon shape.

Then we can invert the definition of interior and exterior by changing the sign of the SDF, and get a perfectly clean SDF inside the room. Of course, the exterior of the room now has the wrong SDF, but assuming our game or film scene happens inside the room, we will be totally fine (except, again, in the computation of some of the surface normals). This is an example of the idea in action:



Filling up the exterior - Incorrect Exterior SDF



Filling up the interior - Incorrect Interior SDF



Correct SDF

Idea 3

Of course, since we can in principle generate both correct internal and external SDFs, and we can also exactly discern between interior and exterior volumes, we can pick the appropriate exact SDF as needed. This comes at the cost of double modeling cost (in artist time) and storage.

Idea 4

In 2D, there's one thing one can do - model the boundary of the object as a series of line segments, circle arcs and quadratic bezier segments, all of which have exact SDFs. If the shape is a closed loop, determining interior and exterior areas is really simple (a cross product per segment suffices). Therefore we can produce arbitrary closed shapes with correct SDF both inside and outside the shape. This idea of using a contour SDF is how the ground-truth pictures in this article were produced. You can see one possible implementation of this 2D contour-SDF modeling in this shader: <https://www.shadertoy.com/view/3t33WH>.

From there, one can use the elongation or revolution operators described in the [3D SDF index page](#) to generate exact 3D shapes with correct interior and exterior distances!

This is so far my favorite technique since, while not general, it's the closest to being it.

Idea 5

One last idea is what I hinted to at the beginning of the introduction of the article - instead of only using basic (often convex) SDF primitives, one can start embracing more complex (often concave) yet exact SDF primitives. You'll find many of such SDFs in my [2D SDF index page](#) and on my [3D SDF index page](#). The reason I've been deriving and collecting them is exactly this one.

As an example, the [cross SDF](#) could have been implemented as the union/min() of two boxes, which would produce the incorrect interior SDF. However, this cross SDF implementation not only implements the correct interior (and exterior) distance field, but **also** is faster, since it can use the symmetry of the problem to evaluate a single box-like formula. This is also the case with `sdHorseshoe()`, `sdOctagon()` and many others.

Similarly, shapes that could be made with basic primitives combined with subtractions and intersection operations also suffer from incorrect distances (exterior distances in that case). And many of them can be made correct everywhere and more performant by implementing the final shape directly as an SDF, such as in the case of `sdArc()`, `sdPie()`, `sdVesica()`, etc.